МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №4

з курсу: „ Обробка надвеликих масивів даних”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

КИЇВ
2025

Ціль роботи: Навчитись налаштовувати систему потокової обробки та менеджменту даних та інтегрувати її із «виробниками» та «споживачами» інформації. Навчитись працювати з застуванням Apache Kafka. Зрозуміти принципи роботи Apache Kafka та можливості застосування для обробки потоків великих даних

Завдання

1. Запуск Apache Kafka в Docker: Ваше перше завдання - розгорнути Apache Kafka в середовищі контейнеризації за допомогою Docker. Це дозволить вам створити ізольоване середовище для роботи з Kafka та дослідити його можливості.

2. Створення Виробника Даних: Напишіть програму, яка виступатиме у ролі "Виробника" даних. Ця програма буде генерувати та передавати дані через Apache Kafka до теми (topic) на сервері Kafka. Вам належить визначити формат та зміст даних, що генеруються.

3. Створення Споживача Даних: Напишіть програму, яка виступатиме у ролі "Споживача" даних. Ця програма повинна підписатися на тему Kafka та зчитувати дані, які надходять від "Виробника". Вам належить обробити ці дані та вивести їх на екран або зробити іншу корисну обробку.

4. Контрольні Запитання: Ваша остання задача - дати відповіді на контрольні запитання, що стосуються принципів роботи Apache Kafka, та поділитися своїми враженнями від роботи з цією системою та контейнеризацією.

Варіант: 21%9 = 3

Номер	Тип використання Kafka	Тип компанії	К-сть тем (Topic)	К-сть розділів (Partition)
3	3	Теплова електростанція	2	1

Табл.1 Варіант

Хід роботи

1) Запуск Apache Kafka в Docker: Ваше перше завдання - розгорнути Apache Kafka в середовищі контейнеризації за допомогою Docker. Це дозволить вам створити ізольоване середовище для роботи з Kafka та дослідити його можливості.

Запуск Apache Kafka у середовищі Docker забезпечує ізольоване та контрольоване середовище для роботи з розподіленою системою обробки потоків даних. Для розгортання було використано файл docker-compose.yml, у якому визначені два контейнери: Zookeeper та Kafka, оскільки Zookeeper відповідає за координацію кластеру та керування метаданими брокера Kafka.



```yaml
version: '3.8'

services:

  zookeeper:
    image: wurstmeister/zookeeper
    container_name: zookeeper
    ports:
      - "2181:2181"

  kafka:
    image: wurstmeister/kafka
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_ADVERTISED_HOST_NAME: localhost
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
```

Рис. 1 docker-compose yml.

Файл docker-compose.yml описує два сервіси: Zookeeper та Kafka, які необхідні для роботи системи обміну повідомленнями Apache Kafka. Docker Compose дозволяє автоматично розгорнути та зв’язати ці сервіси між собою

У секції services визначені всі контейнери, які запускаються у складі цієї інфраструктури.

Сервіс Zookeeper (зберігає метадані Kafka і координує роботу брокера):

1. image: wurstmeister/zookeepe використовується готовий Docker-образ Zookeeper.
2. container_name це ім’я контейнера у Docker Dashboard
3. ports: "2181:2181" це відкриття порту Zookeeper для Kafka.

Сервіс Kafka:

1. image: wurstmeister/kafka (використовується офіційний Docker-образ Kafka)
2. container_name: kafka, ім’я контейнера, яке відображається в Docker Desktop.

3. ports: "9092:9092" Відкриває порт Kafka (9092 основний порт для клієнтів (Producer/Consumer), саме через нього твої Python-програми підключаються до Kafka.

Параметри environment:

1. KAFKA_ADVERTISED_HOST_NAME: localhost - адреса брокера, яку Kafka повідомляє клієнтам (Producer/Consumer).
2. KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181 в казує Kafka, де знаходиться Zookeeper (ім’я контейнера zookeeper, порт 2181)

Після виконання команди docker-compose up обидва сервіси запускаються автоматично, відкриваючи відповідні порти: 2181 для Zookeeper та 9092 для Kafka. Таким чином, Docker створює локальне середовище, у якому брокер Kafka повністю готовий приймати, зберігати та розповсюджувати повідомлення між клієнтами.

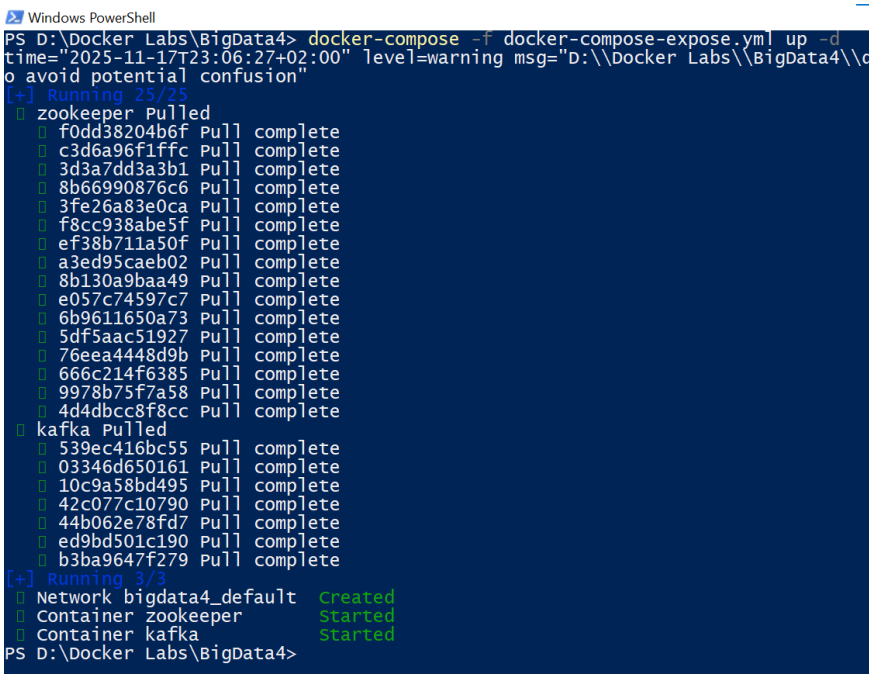


Рис.2 Виконання команди docker-compose up

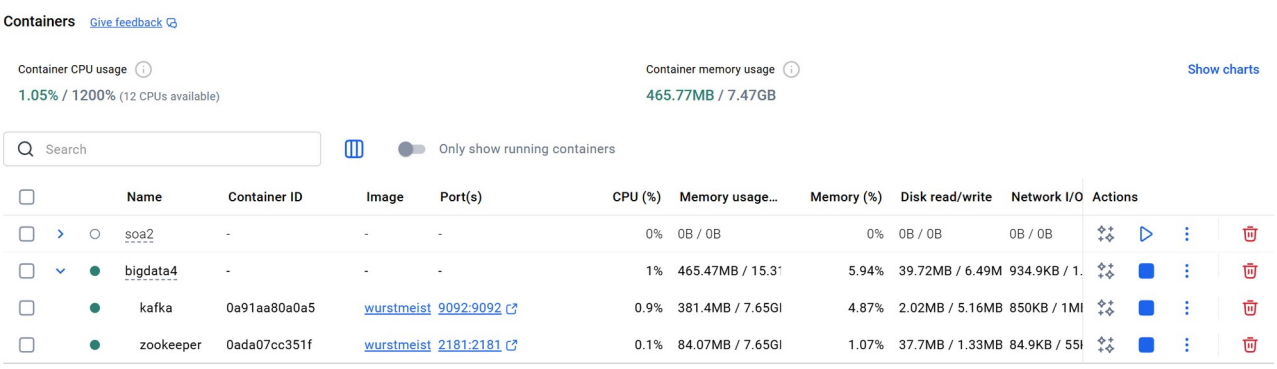


Рис.3 Результат команди

2) Створення Виробника Даних: Напишіть програму, яка виступатиме у ролі "Виробника" даних. Ця програма буде генерувати та передавати дані через Apache Kafka до теми (topic) на сервері Kafka. Вам належить визначити формат та зміст даних, що генеруються.

Далі реалізуємо программу-виробника, яка генерує команди для керування турбіною та відправляє їх у тему Kafka. Основним завданням Producer є формування повідомлень та їх доставка до брокера Apache Kafka, який працює у Docker-контейнері.

```
producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)
```

Рис. 4 Ініціалізація Producer

bootstrap_servers це адреса брокера Kafka (localhost:9092), який ми розгорнули в Docker, а value_serializer визначає спосіб серіалізації повідомлень перед відправленням: повідомлення перетворюється у JSON і кодується у UTF-8.

```
commands = [
    "start turbine",
    "stop turbine",
    "increase power",
    "decrease power",
    "temperature status"
]
```

Рис.5 Формування переліку команд

Це набір тестових команд, які Producer буде послідовно надсилати у Kafka (моделювання роботи системи керування турбіною)

```
for cmd in commands:
    message = {"command": cmd, "timestamp": time.time()}
    producer.send( topic: "command-topic", message)
    print("Sent command:", message)
    time.sleep(1)
```

Рис.6 Цикл відправки даних у Kafka

Тобто ми формуємо повідомлення, потім воно відправляється у топік. Також наявна затримка, вона робить лог зрозумілішим і моделює реальний потік даних

```
Producer is running
Sent command: {'command': 'start turbine', 'timestamp': 1764193190.0207047}
Sent command: {'command': 'stop turbine', 'timestamp': 1764193191.1393356}
Sent command: {'command': 'increase power', 'timestamp': 1764193192.1486473}
Sent command: {'command': 'decrease power', 'timestamp': 1764193193.1551967}
Sent command: {'command': 'temperature status', 'timestamp': 1764193194.1613595}
```

Рис.7 Результат роботи коду

3) Створення Споживача Даних: Напишіть програму, яка виступатиме у ролі "Споживача" даних. Ця програма повинна підписатися на тему Kafka та зчитувати дані, які надходять від "Виробника". Вам належить обробити ці дані та вивести їх на екран або зробити іншу корисну обробку.

Тут реалізуємо консьюмера, який підписується на тему Kafka, отримує команди, обробляє їх та відправляє відповідь у іншу тему. Consumer відіграє роль контролера турбіни, який реагує на команди, отримані від Producer.

```
consumer = KafkaConsumer(
    *topics: "command-topic",
    bootstrap_servers='localhost:9092',
    auto_offset_reset='earliest',
    value_deserializer=lambda x: json.loads(x.decode('utf-8'))
)
```

Рис.8 Підключення до Kafka

Тут у нас прописанийй консьюмер, у якому наявні наступні параметри: command-topic (назва теми, з якої споживач буде читати повідомлення), потім bootstrap_servers (localhost:9092 адреса Kafka, запущеної в Docker), далі auto offset_reset = earlies (читати повідомлення з самого початку, навіть якщо споживач запускається після Producer) і value_deserializer функція, яка приймає байтові дані від Kafka, перетворює їх у JSON-рядок,і повертає Python-об'єкт.

```python
producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)
```

Рис.9 Ініціалізація Producer для відправки відповідей

Consumer тут не лише отримує команди, а і повертає результати обробки, тому додатково створюється Producer, який надсилає відповіді у тему response-topic.

```python
for msg in consumer:
    command = msg.value["command"]
    print("\nReceived:", command)

    if command == "start turbine":
        response = "Turbine started"
    elif command == "stop turbine":
        response = "Turbine stopped"
    elif command == "increase power":
        response = "Power increased by 5%"
    elif command == "decrease power":
        response = "Power decreased by 5%"
    elif command == "temperature status":
        response = "Temperature normal: 520c"
    else:
        response = "Unknown command"

    producer.send( topic: "response-topic",  value: {"result": response})
    print("Sent response:", response)
```

Рис. 10 Обробка отриманих повідомлень та Надсилання відповіді

Consumer працює у безкінечному циклі, кожне отримане повідомлення автоматично десеріалізується а програма дістає поле command (текстову команду, яку надіслав виробник)

```
Turbine Controller is running

Received: start turbine
Sent response: Turbine started

Received: stop turbine
Sent response: Turbine stopped

Received: increase power
Sent response: Power increased by 5%

Received: decrease power
Sent response: Power decreased by 5%

Received: temperature status
Sent response: Temperature normal: 520c
```

Рис. 11 Результат роботи коду

4) Контрольні Запитання: Ваша остання задача - дати відповіді на контрольні запитання, що стосуються принципів роботи Apache Kafka, та поділитися своїми враженнями від роботи з цією системою та контейнеризацією.

1. Що таке “потоки даних” та в чому виникають складнощі при їх обробці?

Потоки даних це безперервні послідовності повідомлень які надходять у реальному часі від різних джерел наприклад сенсорів веб застосунків або серверів. Основна складність обробки потоків полягає у необхідності працювати з великими обсягами даних що надходять без зупинки. Також виникають проблеми з підтримкою високої пропускної здатності забезпеченням низької затримки обробки перевіркою надійності доставки повідомлень і гарантуванням стійкості системи до збоїв при великому навантаженні.

2. Що таке Apache Kafka?

Apache Kafka це розподілена система обробки потокових даних яка працює як швидка черга повідомлень. Kafka дозволяє передавати приймати і зберігати потоки подій з дуже високою швидкістю. Вона є стійкою до збоїв і може працювати у кластері з великою кількістю серверів та клієнтів. Kafka широко використовується в аналітиці потокових даних у логуванні у моніторингу подій або у побудові мікросервісних архітектур.

3. Типи API у Apache Kafka? Структура запису у потоці даних Apache Kafka?

Apache Kafka має кілька основних типів API. Producer API дозволяє програмам надсилати події у теми Kafka. Consumer API дозволяє отримувати події з тем. Stream API забезпечує обробку потокових даних безпосередньо у Kafka. Connector API дозволяє підключати Kafka до зовнішніх систем зберігання або баз даних.

Запис у потоці Kafka складається з трьох основних частин: ключ який використовується для розподілу повідомлень між партиціями значення яке містить самі дані і службові метадані з інформацією про тему номер партиції та зміщення повідомлення.

4. На якому протоколі реалізована комунікація між вузлами системи Apache Kafka?

Комунікація між вузлами Apache Kafka реалізована на основі протоколу TCP. Використання TCP дозволяє гарантувати стабільний обмін повідомленнями з контролем черговості доставки і забезпеченням надійності у мережі.

5. Як досягається підвищений рівень відмовостійкості при обробці даних?

Відмовостійкість у Apache Kafka забезпечується за рахунок реплікації даних. Кожна тема розділена на партиції і для кожної партиції створюється декілька копій на різних серверах кластера. Один сервер виступає лідером а інші є репліками. Якщо лідер виходить з ладу одна з реплік автоматично стає новим лідером що дозволяє системі продовжувати роботу без втрати даних.

6. У якій послідовності «Споживачі» можуть обробляти дані?

Споживачі обробляють дані строго у тій послідовності в якій вони були записані у конкретній партиції теми. Порядок всередині однієї партиції завжди гарантований. Якщо тема має декілька партицій то глобальна послідовність між партиціями не гарантується. Споживач може обробляти дані паралельно якщо він працює у групі споживачів.

7. Яку роль виконують “Partitions of a Topic” Apache Kafka?

Партиції теми виконують важливу роль у масштабуванні продуктивності та забезпеченні відмовостійкості системи. Кожна партиція є окремою логічною чергою записів що дозволяє розподіляти навантаження між кількома споживачами. Партиції дозволяють зберігати дані у різних вузлах і створювати їх репліки. Це дає змогу забезпечити високу швидкість запису і читання а також гарантувати збереження даних під час збоїв.

8. Як реалізована розподіленість Apache Kafka?

Розподіленість Kafka досягається за рахунок кластерної архітектури. Кластер складається з кількох брокерів кожен з яких може зберігати партиції тем. Дані автоматично дублюються між брокерами завдяки механізму реплікації. Споживачі та виробники можуть підключатися до будь якого брокера а Kafka сама керує маршрутизацією повідомлень. Такий підхід дозволяє масштабувати систему горизонтально додаючи нові сервери без зупинки роботи.

9. Як відбувається синхронізація серверів Apache Kafka, «виробників» та «споживачів»?

Синхронізація відбувається через спеціальні механізми Kafka такі як групи споживачів координатор споживачів і контролер кластера. Сервери Kafka синхронізують партиції через реплікацію. Виробники синхронізують надсилання повідомлень використовуючи підтвердження доставки. Споживачі синхронізують своє положення у темі використовуючи зміщення яке Kafka зберігає у координаторі групи. Завдяки цьому Kafka гарантує послідовність та узгодженість при читанні та записі даних.

10. Що таке принцип відсутності копіювання (англ. zero-copy)? Як він використовується у Apache Kafka**.**

Принцип zer-copy полягає у передачі даних між файловою системою та мережею без проміжного копіювання у простір користувача. Це значно знижує навантаження на процесор та підвищує продуктивність обробки великих обсягів даних. У Apache Kafka цей принцип використовується при передаванні повідомлень споживачам. Kafka може відправляти дані безпосередньо з диска у мережевий сокет що робить систему дуже швидкою і ефективною при роботі з великими потоками подій.